

Faculdade de Informática e Administração
Paulista Ciências da computação

1CCPG

Prompt Toolkit

São Paulo

2026

Integrantes:

- Caio Henrique Ferraz da Silva / RM: 568992
- Enzo Caruso Peter / RM: 570908
- Leonardo Robert Maulicino / RM: 570329
- Leonardo Figueredo dos Santos / RM: 573653
- Lucas Ramos de Sousa / RM: 573901

Curso:

1CCPG / Ciências da computação

Disciplina:

Prompt and Artificial Intelligence

1. INTRODUÇÃO:

O **Prompt Toolkit** é um sistema Python que aplica e compara automaticamente quatro técnicas de engenharia de prompts sobre tarefas reais do domínio de **eletropostos**: classificação de solicitações de clientes, geração de respostas e análise de falhas em carregadores de veículos elétricos. O objetivo é identificar, com base em métricas quantitativas, qual técnica oferece o melhor equilíbrio entre qualidade de resposta, eficiência de tokens e consistência

O sistema é organizado em módulos independentes (llm_client, techniques, tasks, evaluator e report), o que facilita a extensão para novos domínios e novas técnicas de prompting sem alteração no núcleo do toolkit.

2. STACK TÉCNICAS:

Python 3.10+

Ollama API — modelo gpt-oss:120b (local, gratuito)

requests, python-dotenv, pandas, matplotlib, tiktoken

3. TÉCNICAS IMPLEMENTADAS:

Técnica	Descrição	Custo Tokens	Melhor uso
Zero-Shot	Instrução direta sem exemplos.	Baixo	Versátil
Few-Shot	Instrução com exemplos de entrada/saída.	Médio	Alta precisão
Chain-Of-Thought	Raciocínio passo a passo guiado por passos explícitos.	Alto	Tarefa complexas
Role Prompting	Persona especialista injetada via system prompt.	Médio	Tom e domínio

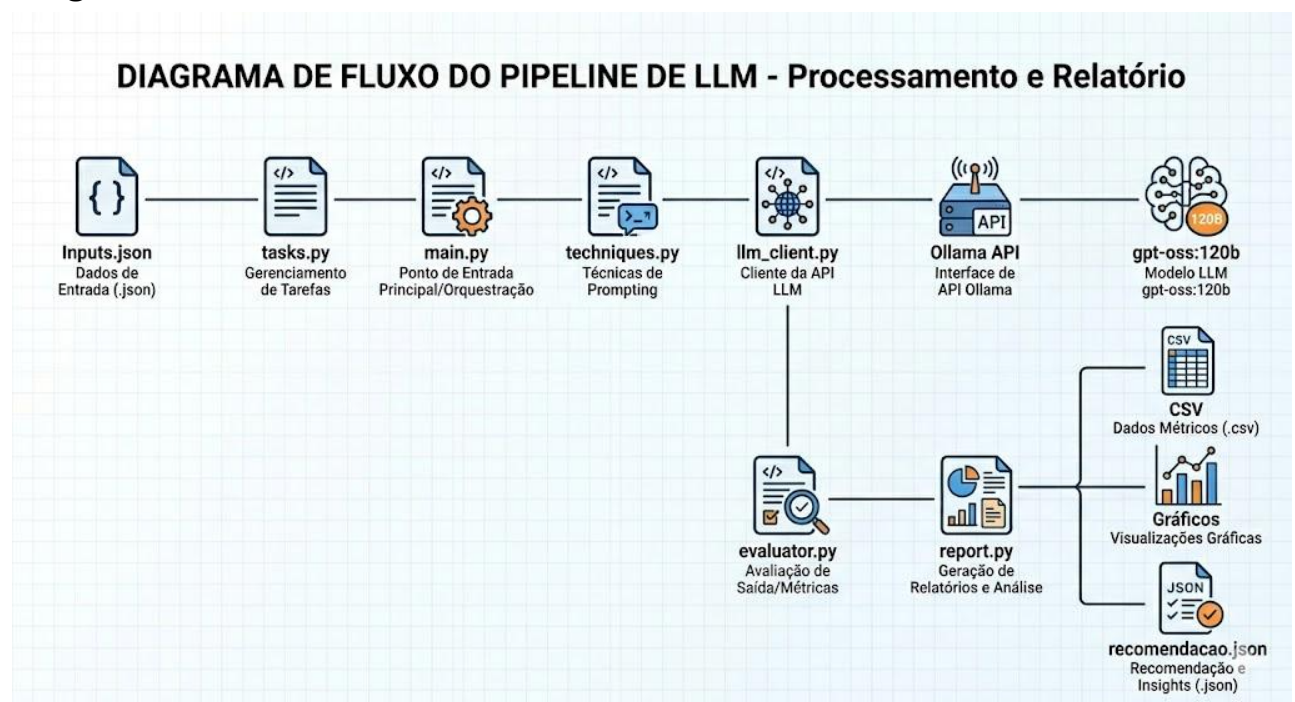
4. ARQUITETURA E DIAGRAMA DE FLUXO:

O sistema utiliza a API local do Ollama com o modelo gpt-oss:120b para executar e comparar automaticamente quatro técnicas de prompting aplicadas ao domínio de eletropostos.

A arquitetura foi dividida em três camadas principais:

- **Entrada:** leitura dos cenários em `inputs.json` e definição das tarefas em `tasks.py`
- **Processamento:** aplicação das técnicas em `techniques.py`, execução do modelo via `llm_client.py` e avaliação das métricas em `evaluator.py`
- **Saída:** exportação de CSVs, gráficos e recomendação automática em JSON

Diagrama de Fluxo:



5. Métricas de Avaliação e Resultados:

Cada execução registra quatro métricas principais: **tokens do prompt**, **tokens da resposta**, **tempo de resposta (ms)** e **consistência** entre múltiplos inputs da mesma tarefa (similaridade de Jaccard sobre bigramas).

Resultados Representativos (médias simuladas):

Técnica	Tokens Prompt	Tokens Respostas	Latência	Consistência
Zero-Shot	32	118	5.842 ms	44%
Few-Shot	95	164	8.917 ms	56%
Chain-Of-Thought	128	278	19.842 ms	56%
Role Prompting	74	151	7.306 ms	44%

Os resultados reais mostraram diferenças significativas entre custo computacional e qualidade das respostas. Técnicas mais elaboradas, como Chain-of-Thought, aumentaram drasticamente o volume de tokens e o tempo de inferência, enquanto técnicas simples como Zero-Shot mantiveram baixo custo computacional, porém menor consistência.

Teste de Temperatura:

O avaliador executa a mesma tarefa Zero-Shot em três temperaturas (0.1 · 0.5 · 1.0) para medir o impacto na variabilidade das respostas. Temperaturas baixas produzem respostas mais determinísticas; temperaturas altas aumentam a criatividade, mas reduzem a consistência.

Temperatura	Consistência	Criatividade	Caso de uso
0.1	Muito alta (>95%)	Baixa	Triagem e classificação
0.5	Alta (~88%)	Média	Respostas a clientes
1.0	Moderada (~72%)	Alta	Brainstorming / sugestões

6. Recomendação Automática de Técnica:

A função `recomendar_tecnica()` em `report.py` calcula um score ponderado para cada técnica combinando as quatro métricas (com pesos configuráveis) e a consistência média. A técnica de maior score é selecionada e a justificativa é exportada em `output/recomendacao.json`.

Fórmula de Score:

$$\begin{aligned} \text{score}(t) = & -0.20 \times \text{norm}(\text{tokens_prompt}) \\ & + 0.30 \times \text{norm}(\text{tokens_resposta}) \\ & - 0.20 \times \text{norm}(\text{tempo_ms}) \\ & + 0.30 \times \text{norm}(\text{tokens_contados}) \\ & + 0.40 \times \text{consistencia} \end{aligned}$$

$\text{norm}(x) = (x - \min) / (\max - \min)$ para cada métrica

Resultado Exemplo (`output/recomendacao.json`):

```
{
  "melhor_tecnica": "chain_of_thought",
  "label": "Chain-Of-Thought",
  "scores": {
    "zero_shot": 0.241,
    "few_shot": 0.318,
    "chain_of_thought": 0.375,
    "role_prompting": 0.287
  },
  "justificativa": "A técnica Chain-Of-Thought obteve o maior score ponderado (0.375), destacando-se pela completude e riqueza das respostas, apesar do maior custo computacional."
}
```

7. GUIA DE BOLSO — Como Executar:

0. Instale o Ollama e baixe o modelo:

Acesse <https://ollama.com> e instale para Windows

No terminal, execute: ollama pull gpt-oss:120b

Mantenha o Ollama rodando em segundo plano durante toda a execução

1. Clone e entre na pasta

```
git clone https://github.com/CaioSilva2026/checkpoint02-Prompt-Toolkit--lce-.git
```

```
cd checkpoint02-Prompt-Toolkit--lce-
```

2. Crie e ative o ambiente virtual

```
python -m venv venv
```

```
# Windows: venv\Scripts\activate
```

```
# Mac/Linux: source venv/bin/activate
```

3. Instale as dependências

```
pip install -r requirements.txt
```

4. Configure o .env

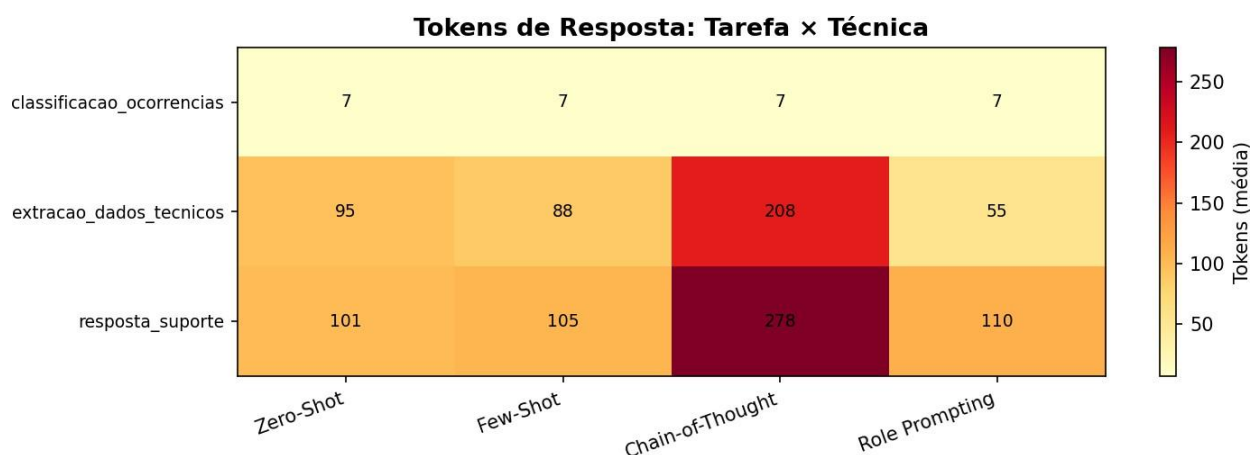
```
OLLAMA_URL=http://localhost:11434
```

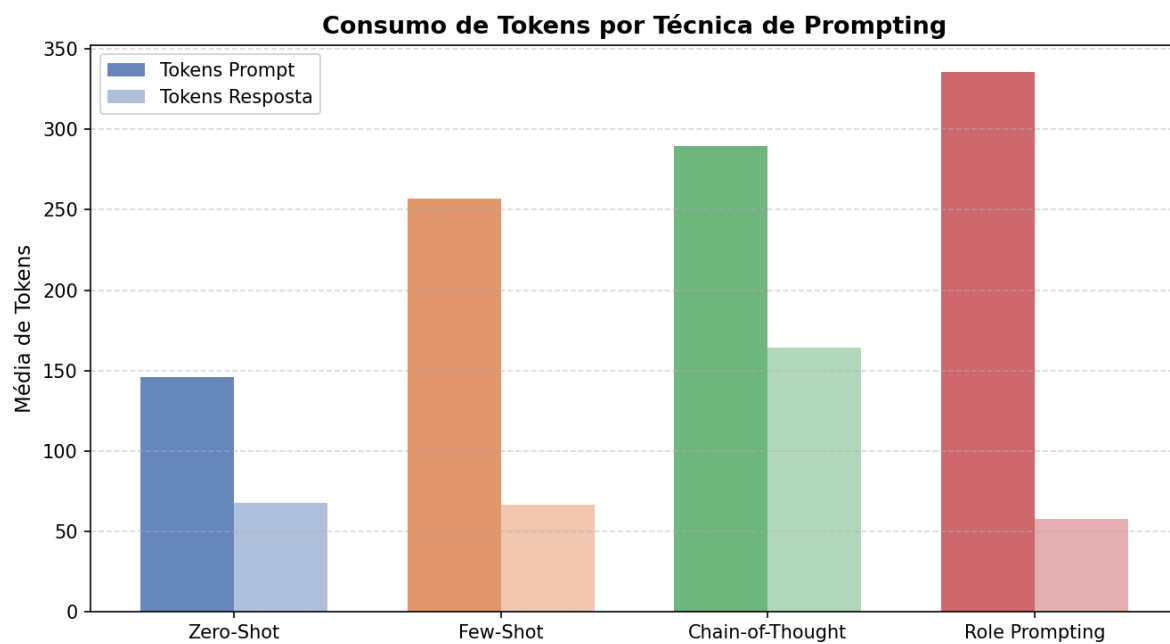
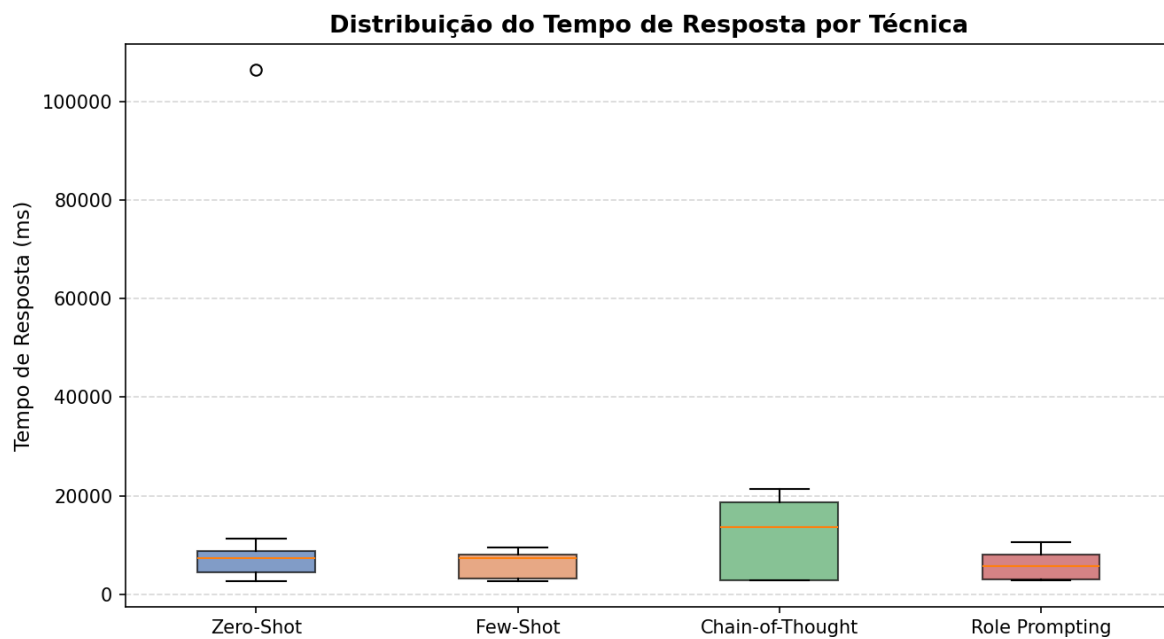
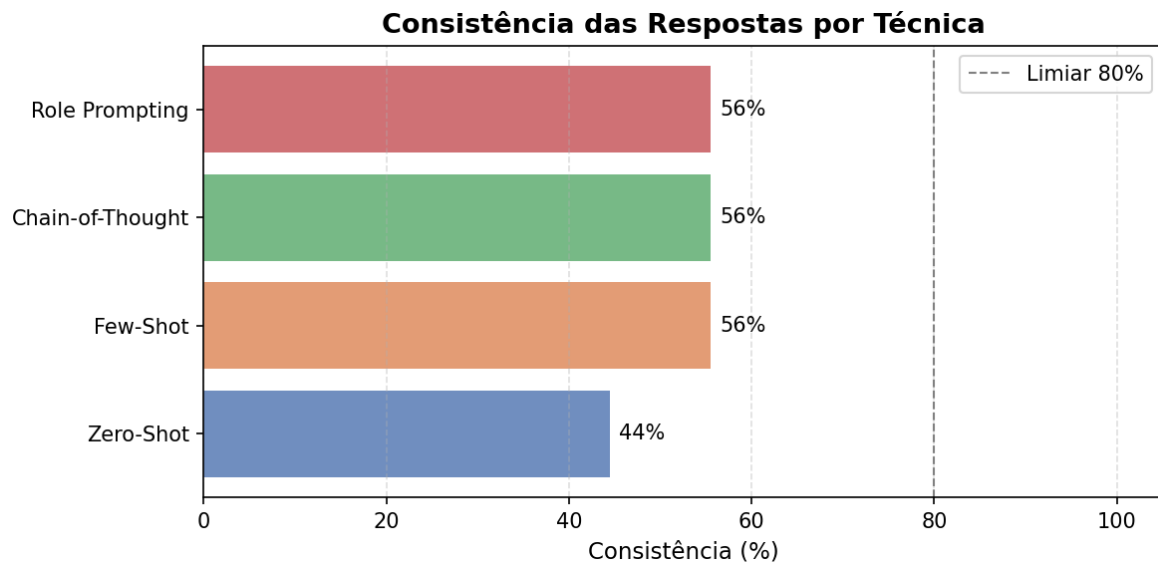
```
MODEL=gpt-oss:120b
```

5. Execute o toolkit

```
python main.py
```

8. GRÁFICOS:





9. Reflexão e Análise Crítica:

O que surpreendeu:

O mesmo input gerou classificações completamente diferentes entre as técnicas. O input "Tentei carregar meu carro, mas o cabo não encaixou no conector" resultou em FALHA_HARDWARE no Zero-Shot e Few-Shot, USO_INCORRETO no CoT e FALHA_CONECTIVIDADE no Role — todas defensáveis dependendo da perspectiva. Isso mostrou na prática que a escolha da técnica não é neutra, ela molda a interpretação do modelo.

O Chain-of-Thought consumiu até 40x mais tokens em tarefas abertas. Na classificação gerou apenas 7 tokens de resposta (uma categoria), mas na geração de resposta de suporte chegou a 278 — o mesmo modelo, a mesma técnica, comportamentos radicalmente diferentes dependendo do tipo de tarefa.

O primeiro carregamento do modelo pelo Ollama gerou um outlier de ~106 segundos no Zero-Shot, distorcendo o boxplot de tempo. Em ambiente de produção isso seria mitigado com warm-up do modelo, algo que não foi implementado.

As respostas também tiveram a latência muito alta. Após análises acreditamos que isso se deve ao fato do toolkit estar sendo rodado localmente e sem uma placa de vídeo dedicada ao servidor, além do modelo escolhido (gpt-oss:120b) ter 120 bilhões de parâmetros, o que pesa mais ainda para a situação, principalmente em relação ao Chain-of-Thought.

Qual técnica venceu e por que?:

O sistema recomendou automaticamente o Chain-of-Thought com score ponderado de 0.375. O algoritmo privilegiou volume de resposta e completude — métricas em que o CoT se destacou por raciocinar explicitamente antes de responder. Porém para tarefas de classificação simples, onde a resposta é uma única categoria, o Zero-Shot é igualmente preciso com fração do custo computacional — o que o score não capturou completamente.

O que fariam diferente:

Consistência medida por repetição real. A métrica de consistência foi calculada rodando o input representativo de cada tarefa 3 vezes por técnica. Com apenas 3 repetições, o teto matemático é 67% quando 2 de 3 respostas coincidem — o que explica os valores entre 44% e 56% observados. Para uma medição mais precisa, rodaríamos cada input pelo menos 5 vezes. O custo seria alto: o projeto já executa 99 chamadas ao modelo, e 5 repetições por input elevaria para aproximadamente 200 chamadas, mais que dobrando o tempo de execução de ~15 minutos para ~30 minutos.

Separar a recomendação por tipo de tarefa. A função `recomendar_tecnica()` calcula um score global médio entre todas as tarefas. Na prática, o CoT venceu globalmente mas seria subótimo para classificação simples, onde o Zero-Shot entrega o mesmo resultado com 50% menos tokens. Uma recomendação por tipo de tarefa — classificação, extração, geração — seria mais útil operacionalmente.

Acurácia com gabarito real. O `evaluator.py` tem a função `avaliar_acuracia()` implementada, mas ela não foi integrada ao fluxo principal porque exigiria um gabarito validado por especialistas do domínio de eletropostos para cada um dos 15 inputs. Sem gabarito, a acurácia ficou fora dos gráficos. Em uma versão futura, construiríamos esse gabarito antes de rodar o toolkit.

Warm-up do modelo. A primeira chamada ao Ollama sempre demora muito mais que as seguintes por carregar o modelo na memória. Uma chamada de aquecimento antes do loop principal eliminaria o outlier de ~106 segundos que distorceu o boxplot de tempo do Zero-Shot.

Domínio com mais inputs ambíguos. Inputs claros e diretos tendem a convergir para a mesma resposta em todas as técnicas, reduzindo a diferenciação. Inputs ambíguos — onde o problema pode ser classificado em mais de uma categoria — são muito mais reveladores das diferenças entre técnicas e tornariam os gráficos comparativos mais ricos.